

Evaluation Results Analysis

Dataset: 04_dataset.jsonl (119,040 entities, 11,974 test entities, 470,772 queries)

Overall Results

Retriever	MRR	R@1	R@5	R@10	NDCG@10
Levenshtein	0.094	0.089	0.100	0.105	0.097
Soundex	0.096	0.092	0.101	0.106	0.098
BM25	0.083	0.077	0.091	0.097	0.086
Transliterate	0.565	0.511	0.635	0.681	0.592
Model v2	0.775	0.711	0.859	0.897	0.804

The model dominates all baselines. Transliterate is the strongest baseline at 0.565 MRR, but the model outperforms it by +21 MRR points (+37%).

By Query Type

Retriever	Phonetic MRR	Script MRR	Combined MRR
Levenshtein	0.894	0.014	0.004
Soundex	0.915	0.014	0.004
BM25	0.791	0.014	0.003
Transliterate	0.894	0.684	0.485
Model v2	0.937	0.827	0.738

Key observations:

- Phonetic queries:** All baselines except BM25 perform well (~0.89-0.91 MRR) since Latin edit distance/phonetics work for same-script matching. Model v2 still leads at 0.937.
- Script queries:** Levenshtein/Soundex/BM25 completely fail (~0.014 MRR) — they cannot bridge script boundaries. Transliterate recovers to 0.684. Model v2 reaches 0.827.
- Combined queries** (Latin variant → non-Latin script): The hardest task. Baselines fail entirely (~0.003-0.004). Transliterate gets 0.485. Model v2 achieves 0.738, showing strong cross-script phonetic alignment.

By Script (R@10)

Script	Levenshtein	Soundex	BM25	Transliterate	Model v2
--------	-------------	---------	------	---------------	----------

Script	Levenshtein	Soundex	BM25	Transliterate	Model v2
latin	0.944	0.952	0.891	0.944	0.983
ar	0.016	0.016	0.007	0.659	0.967
ru	0.023	0.023	0.007	0.901	0.974
zh	0.007	0.007	0.018	0.385	0.666
he	0.019	0.019	0.016	0.433	0.954
hi	0.012	0.012	0.002	0.750	0.973
el	0.020	0.021	0.003	0.828	0.967
ko	0.003	0.003	0.017	0.645	0.728
ja	0.005	0.005	0.014	0.598	0.870

Key observations:

- **Arabic, Russian, Hebrew, Hindi, Greek:** Model v2 achieves >0.95 R@10 — near-perfect cross-script retrieval.
- **Chinese (zh) and Korean (ko):** Hardest scripts. Model v2 gets 0.666 and 0.728 R@10 respectively. Transliterate also struggles here (0.385 and 0.645) — Chinese/Korean romanization is highly ambiguous.
- **Japanese (ja):** Model v2 at 0.870 vs transliterate 0.598 — significant improvement from learning phonetic patterns.
- **BM25 on zh/ko:** Slightly higher than other baselines because CJK character n-grams share some surface overlap. Still far below model.

Script Gap Analysis

The "script gap" = difference in R@10 between Latin (phonetic) queries and non-Latin (script) queries:

Retriever	Latin R@10	Avg Non-Latin R@10	Gap
Levenshtein	0.944	0.013	0.931
Soundex	0.952	0.013	0.939
BM25	0.891	0.010	0.881
Transliterate	0.944	0.669	0.275
Model v2	0.983	0.885	0.098

The model closes the script gap from ~0.93 (edit-distance baselines) down to 0.098 — a 10× reduction. This directly answers the research question.

Comparison: Model v0.1 vs Model v2

Metric	v0.1 (58k entities)	v2 (119k entities)
Overall MRR	0.853	0.775
Script R@10	0.891	0.918
Combined MRR	0.000	0.738

v2 overall MRR is lower because the new dataset includes **combined** type queries (3.3M) which are harder. v0.1 had zero combined queries so its overall MRR was inflated. On script queries v2 is better (+0.027 R@10), and v2 now handles combined queries (0.738 MRR) which v0.1 couldn't evaluate at all.

Cross-Analysis: Results vs Dataset EDA

Why combined queries dominate the overall MRR

Combined queries make up **70% of all queries** (3.29M / 4.67M) but are the hardest type (model MRR: 0.738 vs phonetic: 0.937). This is purely a dataset composition effect — v0.1 had zero combined queries, so its overall MRR of 0.853 was inflated relative to v2's 0.775. On script and phonetic queries individually, v2 is equal or better.

Why Greek (el) succeeds despite near-zero Wikidata coverage

Greek has the **lowest Wikidata coverage of any script** (573 entries, 0%) — almost entirely LLM-generated. Yet the model achieves 0.967 R@10 on Greek. The reason: Greek → Latin romanization is nearly bijective. Each Greek character maps to a consistent Latin equivalent ($\alpha \rightarrow a$, $\beta \rightarrow v/b$, $\gamma \rightarrow g$), so LLM-generated transliterations are internally consistent across training examples. The model learns reliable phonetic patterns.

Why Russian succeeds and benefits from Wikidata

Russian has the **highest Wikidata coverage** (9,063 entries, 2%) — reflecting its prevalence as a major world language in Wikidata. Combined with near-bijective Cyrillic → Latin transliteration, Russian achieves the highest non-Latin R@10 at 0.974. The Wikidata ground truth provides additional anchoring beyond LLM generation.

Why Japanese underperforms relative to Wikidata coverage

Japanese has the second-highest Wikidata coverage (7,533 entries, 1%), yet model R@10 is only 0.870 — lower than Arabic (0.967), Hebrew (0.954), and Greek (0.967) despite their near-zero Wikidata coverage. The cause is **romanization ambiguity**: Japanese has three competing romanization systems (Hepburn, Kunrei-shiki, Nihon-shiki) plus the complexity of kanji readings. The LLM generates inconsistent romanizations across training examples, making the embedding space noisier for Japanese.

Why Chinese and Korean are the hardest scripts

Both zh and ko have near-zero Wikidata coverage (zh: 3,579 = 1%, ko: 1,485 = 0%) and suffer from severe romanization ambiguity:

- Chinese: A single character like "張" maps to Zhang/Chang/Cheung/Djang across dialects and romanization conventions
- Korean: "박" maps to Park/Pak/Bak/Bach depending on convention

The model sees conflicting phonetic mappings for the same entity during training, producing noisy embeddings. Transliterate also fails here (zh: 0.385, ko: 0.645 R@10) because ICU's rule-based system produces only one canonical romanization that may not match the LLM variant used as the query.

Why edit-distance baselines fail completely on script/combined queries

Levenshtein/Soundex operate on raw character sequences. The UTF-8 byte distance between "Schwarzenegger" and "שוורצנגר" is maximal — no shared bytes. Even if the phonetic content is identical, the byte-level representation shares nothing across scripts. This explains the cliff from phonetic MRR ~0.89 to script MRR ~0.014 for these baselines.

Why BM25 behaves differently on CJK scripts

BM25 slightly outperforms Levenshtein/Soundex on zh and ko (R@10: 0.018/0.017 vs 0.007/0.003). This is because BM25 operates on character n-grams — CJK characters in the query and corpus may share some overlap if the same character appears in both. This is coincidental rather than phonetic matching, but gives a marginal lift. Still far below the model.

Why transliterate is the strongest baseline overall

Transliterate bridges the script gap by converting everything to Latin first via ICU's rule-based transliterator, then computing edit distance on comparable strings. This is why its script MRR (0.684) is 49× higher than Levenshtein's (0.014). However it still fails on combined queries relative to the model (0.485 vs 0.738) because it applies a fixed rule-based mapping that cannot learn the phonetic consistency of LLM-generated variant pairs.

FAISS Index Ablation

Evaluated four FAISS index types on the same model checkpoint (11,974 corpus entities, 470,772 queries):

Index	R@1	R@5	R@10	R@100	Mean Latency	p99 Latency	Index Size
FlatIP (exact)	0.711	0.859	0.897	0.970	0.17ms	0.27ms	11.7MB
IVF-Flat	0.703	0.844	0.878	0.938	0.03ms	0.06ms	11.9MB
HNSW	0.711	0.859	0.896	0.964	0.03ms	0.06ms	14.8MB
IVF-PQ	0.604	0.784	0.833	0.928	0.06ms	0.09ms	0.5MB

Key observations:

- **HNSW is the sweet spot:** Matches FlatIP recall exactly at R@1/R@5 and loses only 0.001 at R@10, while running at 5.7× lower latency (0.03ms vs 0.17ms). Memory overhead is modest (+3.1MB over FlatIP).

- **IVF-Flat trades recall for speed:** 5.7× faster than FlatIP but -1.9% R@10 and -3.2% R@100 — the recall loss from approximate cell assignment is noticeable at higher k.
- **IVF-PQ for memory-constrained deployment:** 96% size reduction (11.7MB → 0.5MB) at the cost of -6.4% R@10 (0.897 → 0.833). Quantization compresses the 256-dim float vectors to sub-byte codes — useful if scaling to millions of entities where index memory dominates.
- **Indexing speed:** All indices build in under 1 second on 11,974 entities — not a bottleneck at this scale. At millions of entities, IVF training time becomes relevant.
- **Why FlatIP is fast here:** 11,974 entities × 256 dims = ~12MB of float32 vectors fits entirely in L3 cache. At this scale, exact search is cheap; approximate methods show their advantage only at larger corpus sizes.

Recommendation: HNSW for production — exact FlatIP recall with graph-based sub-millisecond latency. IVF-PQ if deploying over millions of entities where memory is the constraint.

Limitations

1. **Chinese and Korean:** Both model and transliterate struggle. Romanization of CJK characters is highly ambiguous — multiple valid romanizations exist for the same character.
2. **LLM-generated data dominance:** 99.5% of positives are LLM-generated. Wikidata ground truth covers only 0.5% of entities — evaluation quality depends on LLM transliteration quality.
3. **No combined type in v0.1:** The v0.1 baseline was trained/evaluated on data without combined queries, making direct comparison difficult.
4. **Closed-world evaluation:** Corpus = test entities only (11,974). Real-world retrieval over millions of entities would see lower absolute numbers due to more distractors.